

Name of the Student	Kothakota Rakesh
Reg. No	192525075
GitHub Link	https://github.com/Rakesh192525075/MLA0403.git

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO 2: Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches. (BTL-4)

CO Weightage : 15%

Assessment Tool Weightage: 40%

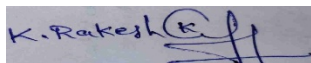
Duration: 90 minutes

Marks: 25

Code of Conduct:

I **Kothakota Rakesh / 192525075** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student

Name of the student: K. Rakesh

Scenario-Based Questions

CO-AT2

QUESTIONS: 05

Total Marks:25

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabelled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

Ans: A hospital has thousands of **unlabelled ECG records**, so supervised learning cannot be directly used because there are no labels such as *normal* or *abnormal*. I would recommend an **Autoencoder**, which is an unsupervised neural network technique.

Justification:

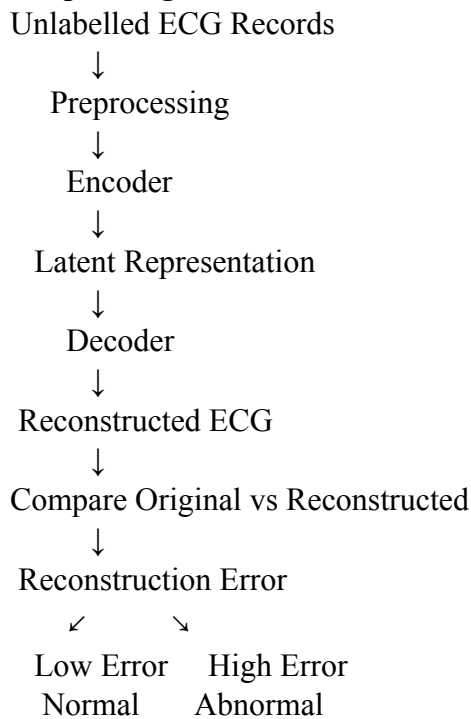
An autoencoder learns the normal patterns in ECG signals by reconstructing the input ECG. When it is trained mainly on normal ECG patterns, it learns to reconstruct normal signals accurately. An abnormal ECG is likely to produce a **higher reconstruction error**, which can be used to detect anomalies.

Workflow

ECG Records → Preprocessing → Autoencoder Training → Reconstruction → Reconstruction Error → Anomaly Detection

1. **Collect ECG data** – Obtain thousands of unlabelled ECG records.
2. **Preprocess the data** – Remove noise, normalize the signals, and divide them into suitable segments.
3. **Encoder** – The encoder compresses the ECG input into a smaller representation called the **latent representation**.
4. **Decoder** – The decoder reconstructs the ECG signal from the latent representation.
5. **Training** – The autoencoder is trained to minimize the difference between the original ECG and reconstructed ECG.
6. **Calculate reconstruction error** – After training, calculate the error between the original and reconstructed ECG.
7. **Set an anomaly threshold** – If the reconstruction error is below the threshold, the ECG is considered **normal**. If it is above the threshold, it is flagged as **abnormal**.
8. **Clinical review** – The detected abnormal ECGs can then be examined by medical professionals.

Simple Diagram



Conclusion:

An **Autoencoder** is appropriate because it can learn useful ECG patterns **without labelled data** and identify abnormal patterns using **reconstruction error**. This makes it particularly useful for anomaly detection in large collections of unlabelled medical records.

2. Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

Ans: An **Autoencoder** can be used to reduce the dimensionality of customer purchase data by learning a **compact representation** of the original high-dimensional data. This compact representation can then be used for **customer segmentation/clustering**.

How it works

Suppose the retail company has customer data such as:

- Number of products purchased
- Total spending
- Purchase frequency
- Category-wise spending
- Discount usage
- Online/offline purchases

- etc.

Instead of directly using all these features for segmentation, an autoencoder compresses them into a smaller number of important features.

Workflow

Customer Purchase Data



Preprocessing



Encoder



Low-Dimensional

Representation

(Latent Space)



Decoder



Reconstructed Data



Compare with Original Data

Steps

1. Input data:

The customer purchase dataset may contain many features, for example, 50 or 100 variables.

2. Encoder:

The encoder gradually reduces the number of dimensions. For example:

100 Features → 50 → 20 → 5 Features

The 5 features form the **latent representation** or compressed representation of each customer.

3. Decoder:

The decoder attempts to reconstruct the original 100 features from these 5 compressed features.

4. Training:

The autoencoder is trained by minimizing the **reconstruction error** between the original customer data and the reconstructed data.

5. Representation learning:

After training, the **encoder part** is used to convert every customer into the learned 5-dimensional representation.

6. Customer segmentation:

These reduced representations can then be given to a clustering algorithm such as **K-Means**.

Customer Data



Autoencoder



5-Dimensional Representation



K-Means Clustering



Customer Segments

For example, the company may discover groups such as:

- **Segment 1:** High-spending customers
- **Segment 2:** Frequent low-value customers
- **Segment 3:** Discount-oriented customers
- **Segment 4:** Occasional customers

Conclusion

An autoencoder performs **dimensionality reduction and representation learning** by compressing high-dimensional purchase data into a smaller latent space while preserving important information. The learned low-dimensional features make **customer segmentation faster, simpler, and potentially more meaningful**.

3. Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

Ans : A facial recognition system may perform poorly with a **shallow neural network** because a shallow network has limited ability to learn complex facial features such as edges, shapes, eyes, nose, and overall face structure.

Recommended Architectural Changes

1. Increase Network Depth

Use a **deeper neural network** with multiple hidden layers instead of only one or two layers.

Input Image



Hidden Layer 1 → Edges



Hidden Layer 2 → Simple Shapes



Hidden Layer 3 → Facial Parts



Hidden Layer 4 → Complex Facial Features



Output Layer → Person Identity

A deeper network can learn **hierarchical features**, from simple patterns in early layers to complex facial patterns in deeper layers.

2. Increase Network Width

Increase the number of neurons/filters in appropriate layers. A wider network can learn more features simultaneously.

For example:

Shallow:

Input → 32 neurons → Output

Improved:

Input → 64 → 128 → 256 → 128 → Output

The exact sizes should be determined through validation rather than assuming that more neurons always improve performance.

3. Use ReLU Activation

Use **ReLU (Rectified Linear Unit)** in the hidden layers:

ReLU is preferred because it is computationally simple and helps neural networks learn nonlinear relationships efficiently.

For the final classification layer, use **Softmax** when the system identifies one person among multiple known classes.

Improved Architecture

Face Image



Input Layer



Dense/Convolutional Layer + ReLU



Deeper Layer + ReLU



Deeper Layer + ReLU



Feature Representation



Softmax Output



Person Identity

Conclusion

The system should be improved by **increasing network depth**, using sufficient **width**, and applying suitable **activation functions such as ReLU** in hidden layers. For a practical facial-recognition system, a **CNN with multiple convolutional layers and ReLU activations** would generally be more suitable than a basic shallow fully connected network because facial images have strong spatial structure.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

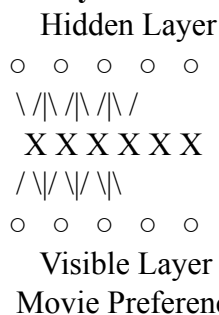
Ans: A **Restricted Boltzmann Machine (RBM)** is an unsupervised neural network that can learn hidden patterns from user–movie interaction data. Therefore, it can be used to recommend movies based on a user's viewing history **without requiring labelled training data**.

How RBM Works

The RBM consists of two layers:

- **Visible layer** → represents observed user movie preferences.
- **Hidden layer** → learns hidden patterns or features in those preferences.

There are connections between the visible and hidden layers, but **no connections within the same layer**.



Workflow

1. Collect viewing history

The streaming service collects user–movie interactions such as:

	Movie A	Movie B	Movie C	Movie D
User 1	1	1	0	0
User 2	1	0	1	1
User 3	0	1	1	0

Here, 1 indicates that the user watched/liked a movie and 0 indicates that the movie was not watched/observed.

2. Input to the RBM

The user's viewing history is provided to the **visible layer**.

3. Learn hidden features

The RBM learns hidden patterns such as:

- Action-movie preference
- Comedy preference

- Sci-fi preference
- Preference for particular actors
- Preference for particular genres

These features are learned automatically from the viewing data.

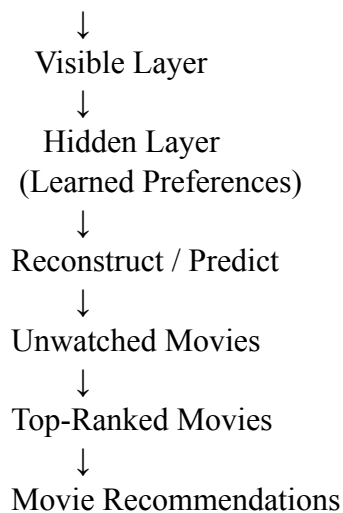
4. Reconstruct preferences

The RBM uses the hidden representation to reconstruct the user's movie preferences and estimate the probability that the user would interact with movies they have not yet watched.

5. Generate recommendations

Movies with high predicted preference scores are recommended to the user.

User Viewing History



Example

If a user frequently watches **science-fiction and action movies**, the RBM may learn hidden features representing these preferences. It can then assign high scores to unseen movies containing similar characteristics and recommend them.

Conclusion

RBM's are useful for recommendation engines because they can **learn hidden user preferences from unlabelled viewing-history data**. By learning the relationship between users' observed movie interactions and hidden preference patterns, an RBM can predict likely interests and recommend suitable movies.

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

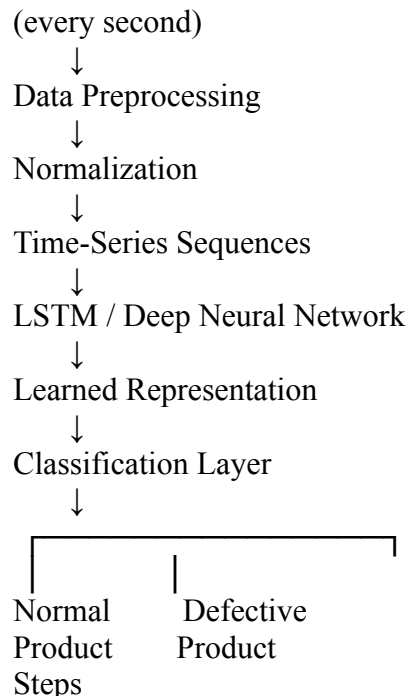
Ans : A manufacturing company can use a deep neural network with representation learning to automatically learn patterns from sensor readings and identify defective products.

Proposed Solution

Since sensor readings are collected every second, the data is sequential/time-series data. A suitable approach is a 1D CNN, LSTM, or CNN-LSTM network. For a syllabus-level solution, an LSTM-based neural network is a good choice because it can learn patterns over time.

Workflow

Sensor Readings



Steps

1. Collect sensor data

Sensors may measure:

- Temperature
- Pressure
- Vibration
- Speed
- Current
- Humidity

For example, readings are collected every second during the manufacturing process.

2. Preprocess the data

Clean missing or noisy values and normalize the sensor readings so that different measurements are on comparable scales.

3. Create time-series sequences

Instead of analyzing each second independently, combine consecutive readings into sequences.

For example:

10 sec → 20 sec → 30 sec → ... → 60 sec

This allows the model to learn how sensor values change over time.

4. Representation Learning

The hidden layers of the neural network automatically learn important features from the raw sensor data, such as:

- Sudden temperature increases
- Abnormal vibration patterns
- Pressure fluctuations
- Unusual combinations of sensor readings

This is called representation learning because the network learns useful representations/features without manually designing them.

5. Classification

The learned representation is passed to a final classification layer.

Learned Features

↓

Fully Connected Layer



Sigmoid / Softmax



Normal / Defective

6. Training

If historical products have defect labels, train the network using normal and defective examples and minimize classification loss.

7. Real-time detection

After training, new sensor readings can be continuously provided to the model. If the model predicts a high probability of a defect, the product can be flagged for inspection.

Example

Suppose a machine normally operates with stable temperature and vibration. If the sensor readings suddenly show:

Temperature ↑

Vibration ↑↑

Pressure fluctuating

the neural network can recognize this learned pattern as being associated with defective products.

Conclusion

A deep neural network such as an LSTM can automatically learn meaningful representations from continuous sensor readings. These learned features can then be used to classify products as normal or defective, enabling early and potentially real-time defect detection without requiring manually engineered features.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure

Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand